AI Tools Hub - 后台管理面板与统计系统需求文档 (Cursor 落地版)

1. 项目概述与开发目标

本文档旨在为 AI Tools Hub 平台设计并构建一个全功能的**总后台管理系统 (Admin Panel)**。该文档专门为 Cursor 等 AI 编程助手编写，包含清晰的页面结构、数据模型和具体的开发提示词。

核心技术栈要求：

- **框架**: Next.js 14 (App Router)
- **UI/样式**: Tailwind CSS + shadcn/ui (保持与前台一致的 Dark Mode 风格)
- **图表库**: Recharts 或 ECharts (用于数据大盘可视化)
- **数据层**: Prisma ORM + PostgreSQL
- **状态管理**: Zustand + React Query

---

2. 全局统计维度与指标定义 (核心需求)

系统必须对前台所有页面以及后台关键模块进行极高颗粒度的数据追踪。所有统计模块必须包含以下四个核心指标：

1. **PV (Page Views - 页面浏览量)**：页面被加载或刷新的总次数。
2. **UV (Unique Visitors - 独立访客数)**：基于设备指纹或浏览器 Cookie (Session ID) 去重后的访客数。
3. **UID (Unique Logged-in Users - 独立登录用户数)**：基于平台真实注册用户 ID 去重后的访问人数（仅统计已登录状态的访问）。
4. **人数 (Total Users / Active Users)**：特定业务场景下的用户基数（如：提交工具的人数、参与评论的人数）。

---

3. 后台管理面板页面结构与功能详细说明

3.1 侧边栏导航 (Sidebar Navigation)

- 📊 **Dashboard (数据大盘)**
- 📈 **Page Analytics (页面流量统计)**
- 🛠️ **Tool Management (工具审核与管理)**

- 👥 **User Management (用户管理)**
- 💬 **Review Management (评论管理)**
- 💰 **Monetization (商业化管理)**
- ⚙️ **System Settings (系统设置)**

3.2 页面详细设计与数据指标

3.2.1 Dashboard (数据大盘) - `/admin/dashboard`

功能描述：全站核心指标的宏观展示面板。

- **顶部数据卡片 (Top Metrics Cards)**:
  - 今日总 PV / 较昨日环比 (+%)
  - 今日总 UV / 较昨日环比 (+%)
  - 今日活跃 UID / 较昨日环比 (+%)
  - 总注册人数 (Total Registered Users)
  - 总收录工具数 (Total Active Tools)
- **流量趋势图表 (Traffic Trend Chart)**:
  - 折线图展示过去 7天/30天/90天 的 PV、UV、UID 走势。
- **待办事项 (Quick Actions)**:
  - 待审核工具数量提示 (Pending Tools)
  - 最新被举报的评论数量提示

3.2.2 Page Analytics (页面流量统计) - `/admin/analytics`

功能描述：细粒度展示前台各个路由页面的流量表现，用于指导 SEO 和内容运营。

- **筛选器**: 日期范围选择器 (Date Range Picker)。
- **数据表格 (Data Table)**:
  - `Page Path` (页面路径，如 `/`, `/tool/chatgpt`, `/compare`)
  - `Page Type` (页面类型：Home, Tool Detail, Category, Submit)
  - `PV` (该页面的浏览量)
  - `UV` (该页面的独立访客)
  - `UID` (该页面的独立登录用户数)
  - `Avg. Time on Page` (平均停留时长)
  - `Bounce Rate` (跳出率)

- **交互**: 支持按 PV 或 UV 降序排列。

3.2.3 Tool Management (工具审核与管理) - `/admin/tools`

功能描述：管理全站 AI 工具，处理开发者提交的审核请求。

- **Tabs**: All / Pending Review / Active / Rejected
- **数据表格**:
  - `Tool ID` & `Logo` & `Name`
  - `Category`
  - `Submitted By` (提交者 UID / 邮箱)
  - `Status` (状态标签)
  - **流量数据**: 该工具详情页的累计 `PV`, `UV`, `UID` 访问数。
  - `Actions`: [Approve] / [Reject] / [Edit] / [Set as Featured]
- **审核抽屉/弹窗 (Review Drawer)**:
  - 展示用户提交的所有字段（网址、描述、截图等）。
  - Reject 时需提供下拉选择拒绝理由（如：网址无效、描述违规、非 AI 工具）。

3.2.4 User Management (用户管理) - `/admin/users`

功能描述：管理全站注册用户，监控用户活跃度。

- **数据表格**:
  - `UID` (内部唯一标识)
  - `Avatar` & `Name` & `Email`
  - `Role` (User / Developer / Admin)
  - `Registration Date` (注册时间)
  - `Last Login` (最后活跃时间)
  - **行为数据**: `Submitted Tools Count` (提交工具数), `Reviews Count` (评论人数/次数)。
  - `Actions`: [Edit Role] / [Ban User] / [Send Email]

3.2.5 Review Management (评论管理) - `/admin/reviews`

功能描述：管理用户产生的 UGC 内容，处理违规评论。

- **数据表格**:
  - `Review ID`
  - `Tool Name` (关联工具)

- `Reviewer` (评论者 UID)
- `Rating` (星级 1-5)
- `Content` (评论内容摘要)
- `Status` (Published / Reported / Hidden)
- `Actions` : [View Full] / [Hide/Delete]

3.2.6 Monetization (商业化管理) - `/admin/monetization`

功能描述：管理开发者的付费推广订单（对应前台 Dashboard 的 $99/month 推广功能）。

- **数据表格**:
  - `Order ID`
  - `Tool Name`
  - `Purchaser UID` (购买者)
  - `Amount` ($99.00)
  - `Payment Status` (Paid / Failed / Pending)
  - `Validity Period` (生效期 - 过期期)
  - **效果数据**: 推广期间带来的额外 `PV` , `UV` , `UID` 点击量。

---

4. 统计数据模型设计 (Prisma Schema 扩展)

为了支撑上述高颗粒度的 PV/UV/UID 统计，需在现有的 Prisma Schema 中增加日志和聚合表：

Plain Text

```
// 1. 细粒度用户行为日志表 (用于流式写入和计算 UV/UID)
model PageViewLog {
  id          String   @id @default(uuid())
  page_path   String   // 访问的路径，如 "/tool/chatgpt"
  session_id  String   // 基于浏览器指纹或 Cookie 生成，用于计算 UV
  user_id     String?  // 如果用户已登录，记录 UID，用于计算 UID 访问数
  ip_address  String?  // 可选，用于防刷
  user_agent  String?
  created_at  DateTime @default(now())

  @@index([page_path, created_at])
  @@index([session_id, created_at])
  @@index([user_id, created_at])
}

// 2. 页面流量每日聚合表 (由定时任务从 Log 表聚合，提升后台查询性能)
```

```
model PageAnalyticsDaily {
  id          String   @id @default(uuid())
  date        DateTime @db.Date // 聚合日期 (YYYY-MM-DD)
  page_path   String   // 页面路径
  pv_count    Int      @default(0) // 当日总浏览量
  uv_count    Int      @default(0) // 当日去重 session_id 数
  uid_count   Int      @default(0) // 当日去重 user_id 数

  @@unique([date, page_path])
}
```

---

5. Cursor 落地开发提示词 (Cursor Prompts)

请将以下 Prompt 直接复制到 Cursor 的 Composer 或 Chat 中执行，即可快速生成代码：

阶段一：初始化后台布局与路由

> **Prompt to Cursor:**
>
> "I need to build the Admin Panel for my Next.js 14 App Router project. Please create a layout wrapper under `app/admin/layout.tsx`. It should include a left Sidebar with navigation links (Dashboard, Page Analytics, Tools, Users, Reviews, Monetization) and a top header with the Admin's avatar. Use Tailwind CSS and `lucide-react` for icons. Ensure it uses a dark theme matching the `shadcn/ui` style. Then, create empty `page.tsx` files for each of these routes."

阶段二：开发 Dashboard 数据大盘

> **Prompt to Cursor:**
>
> "Now, implement the `app/admin/dashboard/page.tsx`. Create a responsive grid with 5 Metric Cards at the top (Total PV, Total UV, Active UID, Total Users, Active Tools). Below the cards, use `recharts` to build a Line Chart component showing 'Traffic Trends (PV, UV, UID)' over the last 30 days. Mock the data for now, but structure the component so it can easily accept real data from a Prisma backend."

阶段三：开发 Page Analytics 统计页

> **Prompt to Cursor:**
>
> "Implement the `app/admin/analytics/page.tsx`. I need a complex Data Table using `shadcn/ui` Table components. The table columns must be: Page Path, Page Type, PV, UV, UID, Avg Time, and Bounce Rate. Add a Date Range Picker filter at the top. Ensure the table

> supports sorting by PV and UV. Please mock a list of 10 different routes (e.g., '/', '/tool/chatgpt', '/submit') with realistic statistical data."

阶段四：开发 Tool Management 审核页

> **Prompt to Cursor:**
>
> "Implement the `app/admin/tools/page.tsx`. Create a Tabs component (All, Pending Review, Active, Rejected). Inside the 'Pending Review' tab, build a table showing Tool ID, Logo, Name, Category, Submitted By (UID), and Traffic Data (PV/UV/UID). Add an 'Actions' column with 'Approve' and 'Reject' buttons. When clicking 'Reject', open a `shadcn/ui` Dialog or Sheet asking the admin to select a rejection reason before confirming."

阶段五：实现 PV/UV/UID 追踪 API (后端)

> **Prompt to Cursor:**
>
> "I need to implement the backend tracking for PV, UV, and UID. Please write a Next.js Route Handler at `app/api/track/route.ts`. It should accept a POST request with `page_path`. It needs to read a `session_id` from cookies (generate a UUID if it doesn't exist to track UV) and extract the `user_id` from the JWT token if the user is logged in (to track UID). Then, write the Prisma query to insert this record into the `PageViewLog` model."

6. 前台全页面埋点与追踪清单 (Tracking Checklist)

为确保后台 `Page Analytics` 能够准确获取各页面的 UV/PV/UID，必须在前台各个页面进行无痕埋点。

页面名称	路由路径	页面类型	触发时机	额外需追踪的核心业务指标 (Event Tracking)
首页	`/`	Home	页面挂载完成	搜索框使用次数、热门分类点击率
工具详情页	`/tool/[slug]`	Tool Detail	页面挂载完成	Visit Website 按钮点击 (Outbound Click)、加入收藏次数、分享次数
分类聚合页	`/category/[slug]`	Category	页面挂载完成	分类下工具的曝光量与点击率

对比工具页	`/compare`	Compare	页面挂载完成	对比工具的添加次数 (对比人数)
提交工具页	`/submit`	Submit	页面挂载完成	表单开始填写人数、成功提交人数
开发者面板	`/dashboard`	Dashboard	页面挂载完成	付费推广(Boost)点击人数
用户主页	`/profile`	Profile	页面挂载完成	修改资料保存次数

---

7. 后台管理面板 API 接口定义 (Admin APIs)

为 Cursor 开发提供的标准 RESTful 接口规范：

7.1 页面流量统计查询

- **Endpoint**: `GET /api/admin/analytics/pages`
- **Query**: `startDate` (YYYY-MM-DD), `endDate` (YYYY-MM-DD), `sortBy` (pv/uv/uid)
- **Response**:

JSON

```
{
  "data": [
    {
      "page_path": "/tool/chatgpt",
      "page_type": "Tool Detail",
      "pv": 15420,
      "uv": 12300,
      "uid": 4500,
      "avg_time_seconds": 125,
      "bounce_rate": 0.35
    }
  ]
}
```

7.2 工具审核状态更新

- **Endpoint**: `PATCH /api/admin/tools/:id/status`

- **Body**: `{ "status": "ACTIVE" | "REJECTED", "reject_reason": "URL_INVALID" }`
- **Response**: `{ "success": true, "message": "Status updated" }`

---

8. 客户端无痕埋点 SDK (Cursor 可直接复制)

请将以下代码提供给 Cursor，用于在 Next.js 前端项目中实现全局的 PV/UV/UID 追踪：

阶段六：实现全局埋点 SDK (Global Tracking Hook)

> **Prompt to Cursor:**
>
> "Please create a custom React hook `usePageTracking.ts` in the `hooks/` directory. It should use `next/navigation`'s `usePathname` and `useSearchParams`. Whenever the route changes, it should send a POST request to `/api/track` with the current `page_path`. To ensure we don't block rendering, use `navigator.sendBeacon` or a non-blocking `fetch`. Then, import and call this hook inside the main `app/layout.tsx` so it tracks every page visit across the entire application automatically."

预期生成的 SDK 代码结构参考：

TypeScript

```
// hooks/usePageTracking.ts
import { useEffect } from 'react';
import { usePathname, useSearchParams } from 'next/navigation';

export const usePageTracking = () => {
  const pathname = usePathname();
  const searchParams = useSearchParams();

  useEffect(() => {
    if (!pathname) return;

    const trackPageView = async () => {
      try {
        const url = `${pathname}${searchParams?.toString() ? `?${searchParams.t

        // 使用 sendBeacon 保证在页面卸载时也能成功发送，不阻塞主线程
        const payload = JSON.stringify({ page_path: url });
        const blob = new Blob([payload], { type: 'application/json' });
        navigator.sendBeacon('/api/track', blob);

      } catch (error) {
        console.error('Failed to track page view:', error);
      }
```

```
    };

    trackPageView();
  }, [pathname, searchParams]);
};
```